

Graph



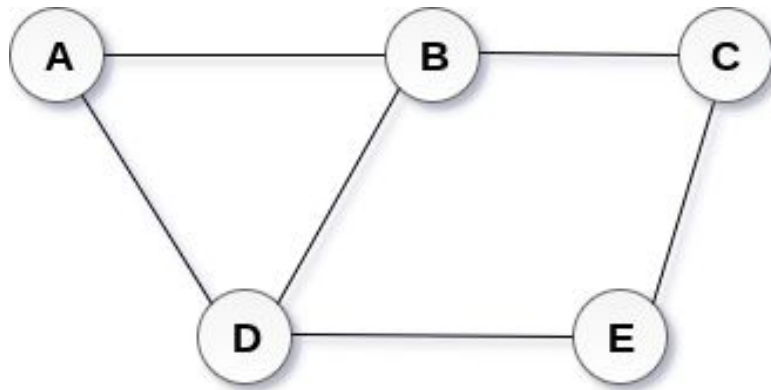
Dr. Md. Atiqur Rahman
Assistant Professor
Dept. of Computer Science & Engineering

Basic structure (Graph)

1. What is Graph?

Definition: A graph G is a **non-linear data structure**, can be defined as an ordered set $G(V, E)$ where $V(G)$ represents the set of vertices and $E(G)$ represents the set of edges which are used to connect these vertices.

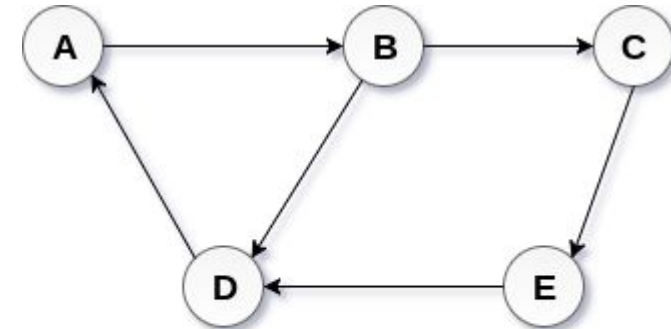
A Graph $G(V, E)$ with 5 vertices (A, B, C, D, E) and six edges ((A,B), (B,C), (C,E), (E,D), (D,B), (D,A)) is shown in the following figure.



Undirected Graph

Directed and Undirected Graph

A graph is **said to be directed** if **edges are associated** with the directions.



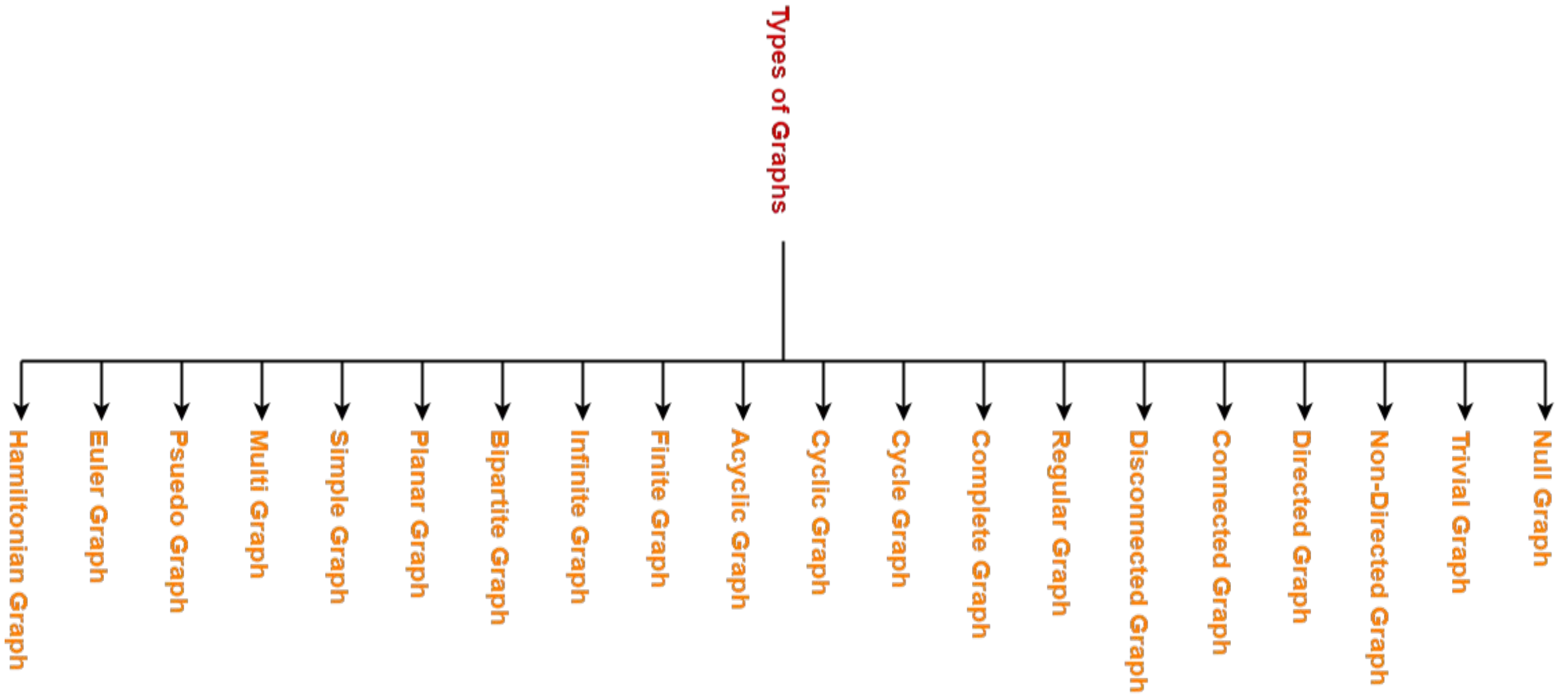
Digraph/ Directed Graph

Basic structure (Graph)

1. What are the differences between Tree and Graph?

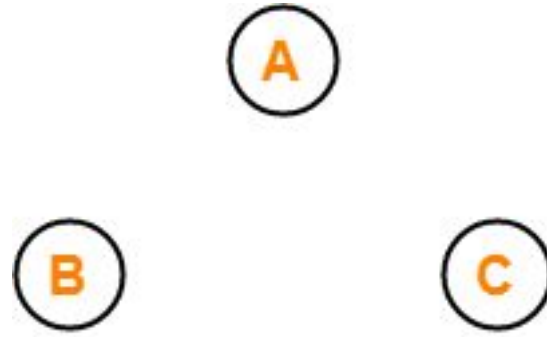
Tree	Graph
✓ A tree is a type of graph that is acyclic (does not contain cycles)	✓ A graph can be acyclic or cyclic.
✓ For N nodes, a tree will have exactly N-1 edges.	✓ For N nodes, a graph can have more or less than N edges.
✓ Each node (except the root node) has only one parent node	✓ A node can have multiple parent or child nodes
✓ A tree is a directed graph towards a root node	✓ It Can be directed or undirected
✓ A tree is a connected graph	✓ It can be connected or disconnected
✓ Generally, less complex than graphs	✓ More complex features

Basic structure (Graph types)



Basic structure (Graph types)

Null Graph: A null graph is defined as a graph which consists only the isolated vertices.



Example of Null Graph

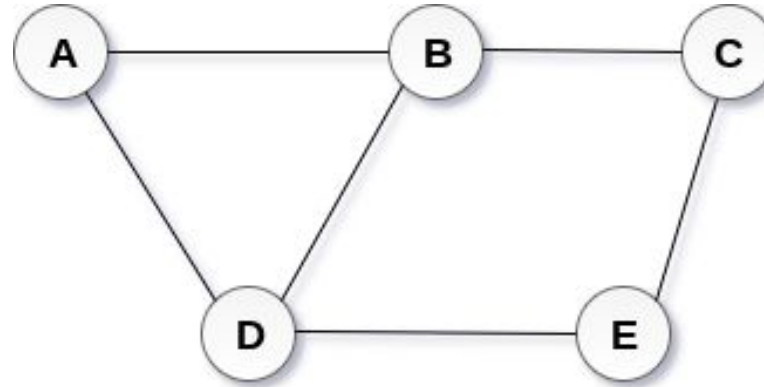
Trivial Graph: A finite graph $G=(V,E)$ is said to be trivial if it has only one vertex. Since such a graph has only one vertex, it does not have any edges.



Example of Trivial Graph

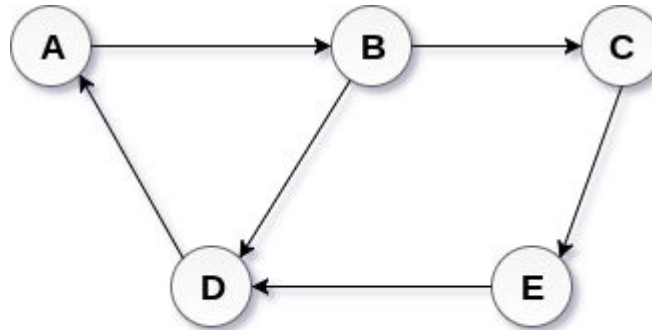
Basic structure (Graph types)

Undirected/Non-directed graph: A graph is said to be undirected if edges are not associated with the directions.



Undirected Graph

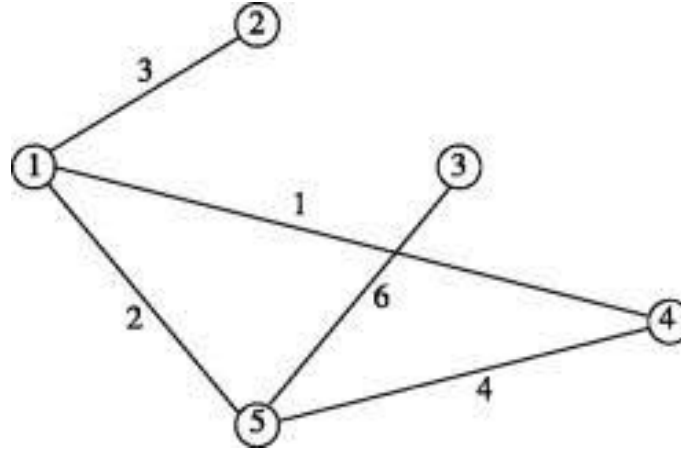
Digraph/ Directed Graph: A graph is said to be directed if edges are associated with the directions.



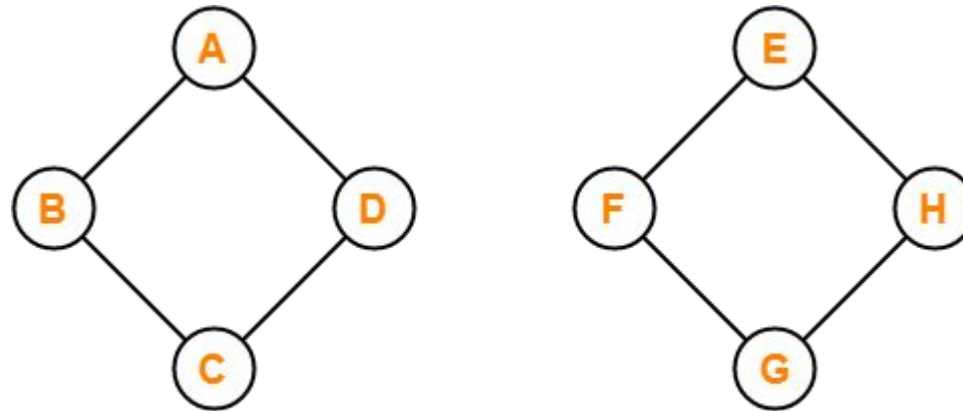
Digraph/ Directed Graph

Basic structure (Graph types)

Connected Graph: A graph is said to be connected if all the nodes are connected.



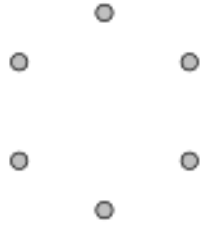
Disconnected Graph: A graph is said to be disconnected if all the nodes are not connected.



Example of Disconnected Graph

Basic structure (Graph types)

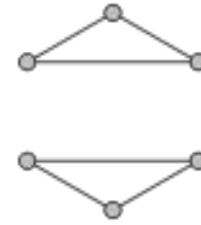
Regular Graph: A graph is said to be regular where each vertex has the same number of neighbors;



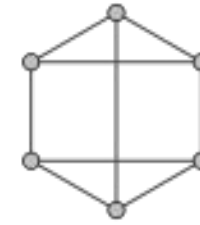
0-regular graph



1-regular graph

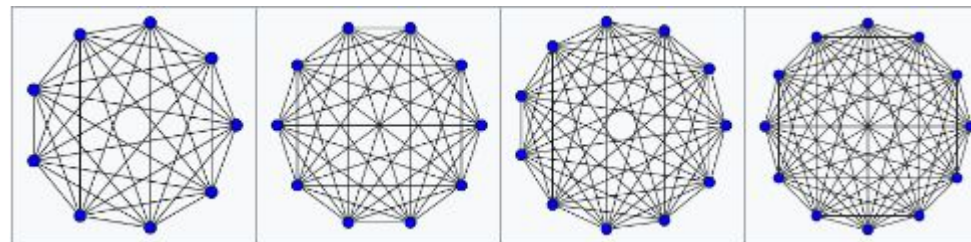
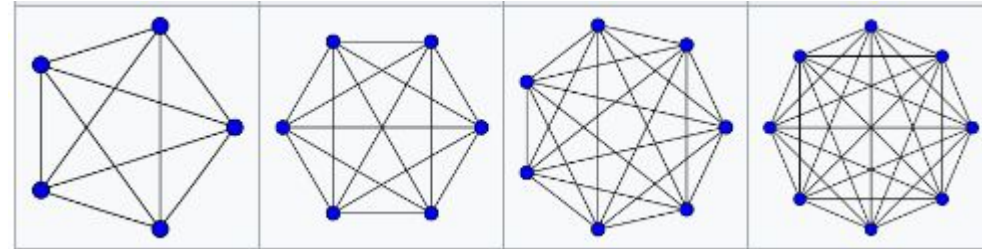
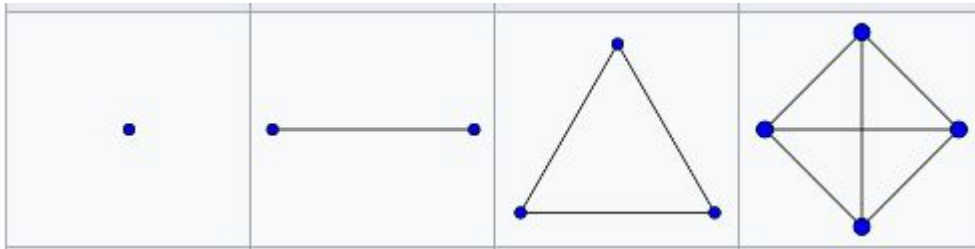


2-regular graph



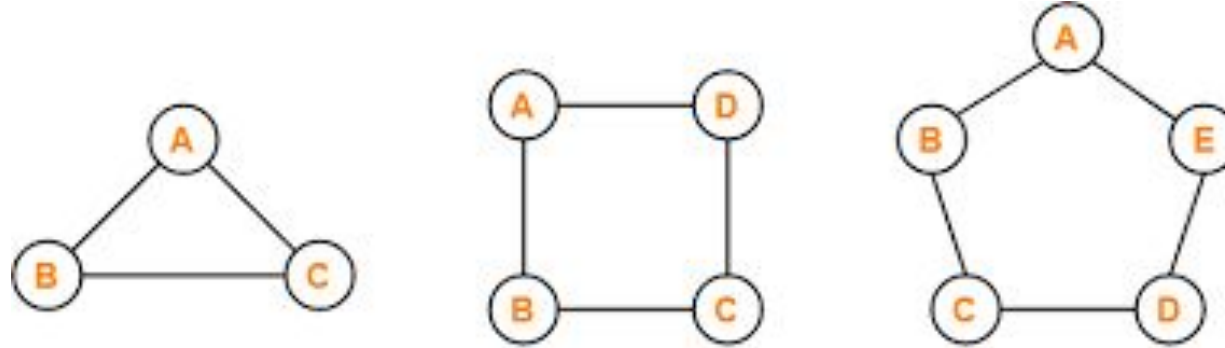
3-regular graph

Complete Graphs: A graph in which each vertex is connected to every other vertex.



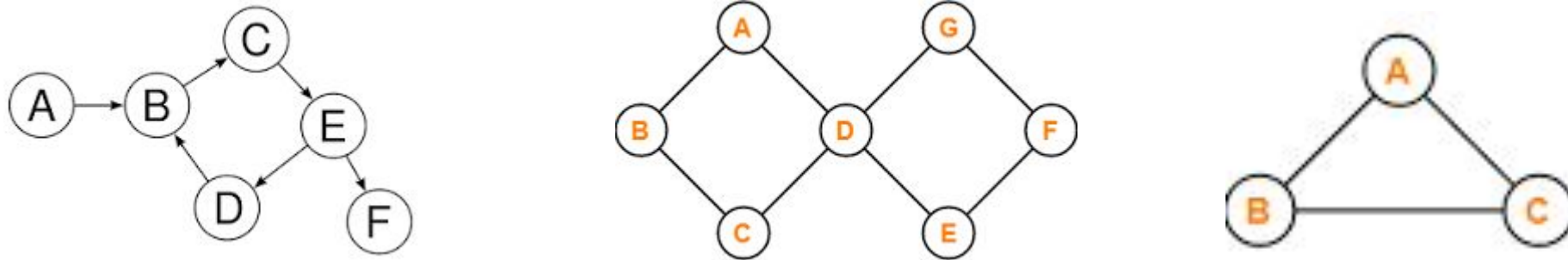
Basic structure (Graph types)

Cycle graph: A cycle graph or circular graph is a graph that consists of a single cycle.



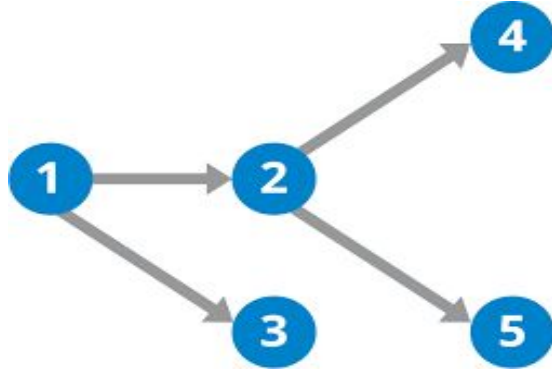
Examples of Cycle Graph

Cyclic graph: A graph that contains at least one cycle.

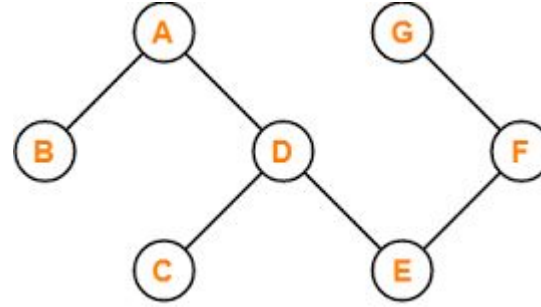


Basic structure (Graph types)

Acyclic graph: An acyclic graph is a graph having no cycles. For example tree.

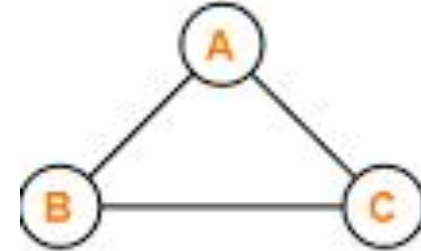
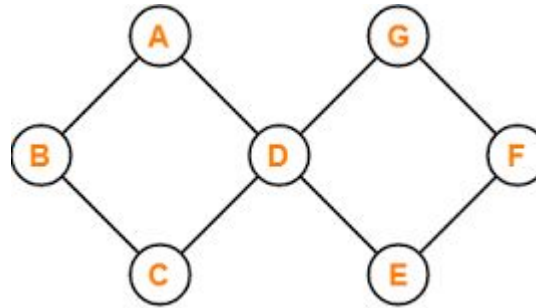
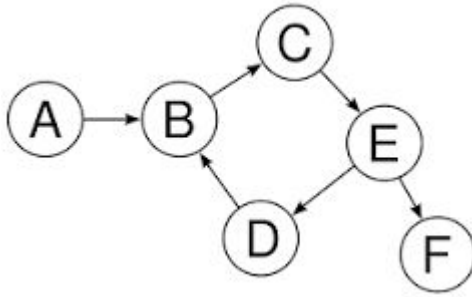


Directed acyclic graph



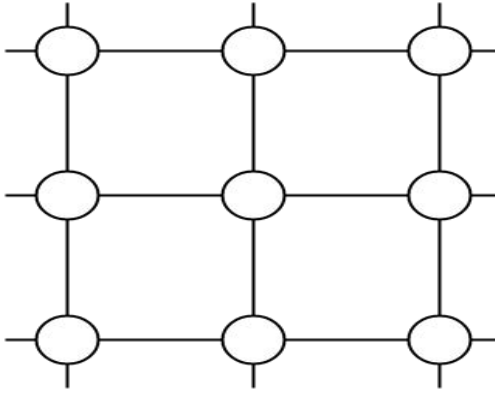
Undirected acyclic graph

Finite graph: A graph with a finite number of edges and a finite number of vertices. All the graphs are finite graph with special case.



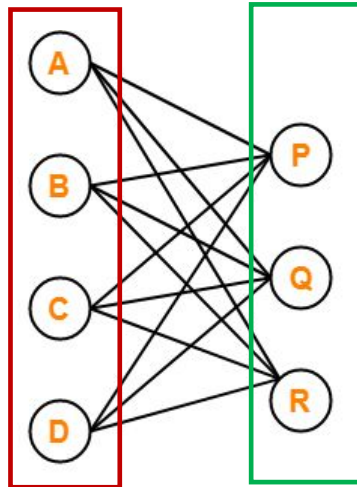
Basic structure (Graph types)

Infinite graph: A graph consisting of infinite number of vertices and edges is called as an infinite graph.



Example of Infinite Graph

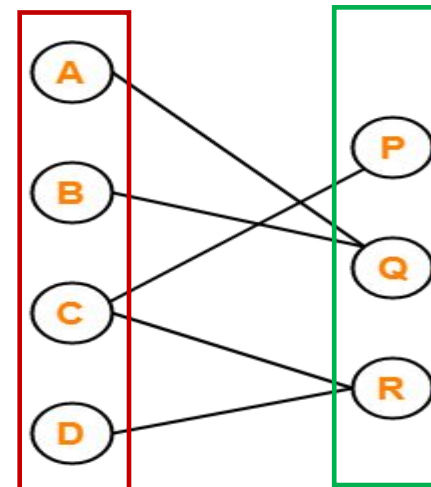
Bipartite graph: A graph $G=(V, E)$ is called a bipartite graph if its vertices V can be partitioned into two subsets V_1 and V_2 such that each edge of G connects a vertex of V_1 to a vertex V_2 .



Example of Complete Bipartite Graph

V_1

V_2



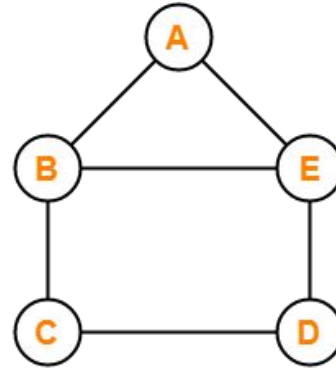
Example of Bipartite Graph

V_1

V_2

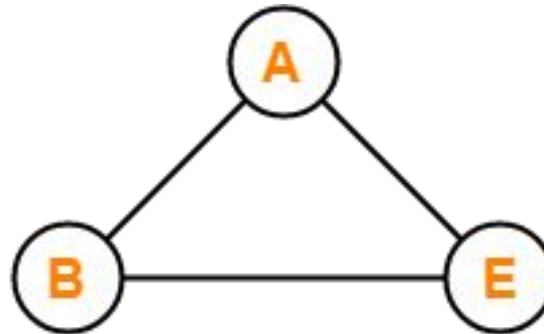
Basic structure (Graph types)

Planar graph: A graph is said to be planar if it can be drawn in a plane so that no edge cross.



Example of Planar Graph

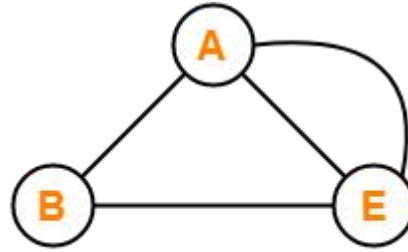
Simple graph: A graph that does not have more than one edge between any two vertices and no self-loop at the same vertex.



Example of Simple Graph

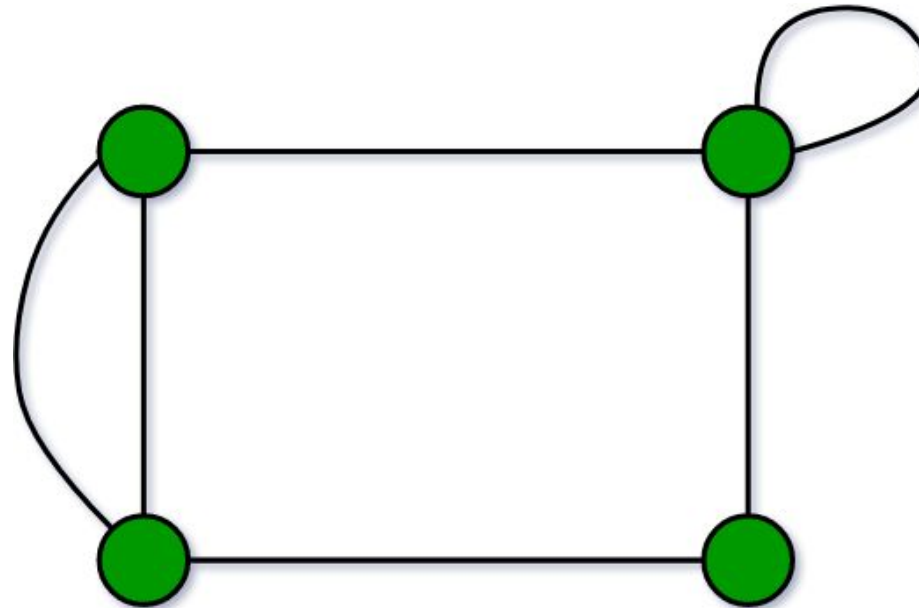
Basic structure (Graph types)

Multigraph: A graph that can have more than one edge between a pair of vertices and no-self loop.



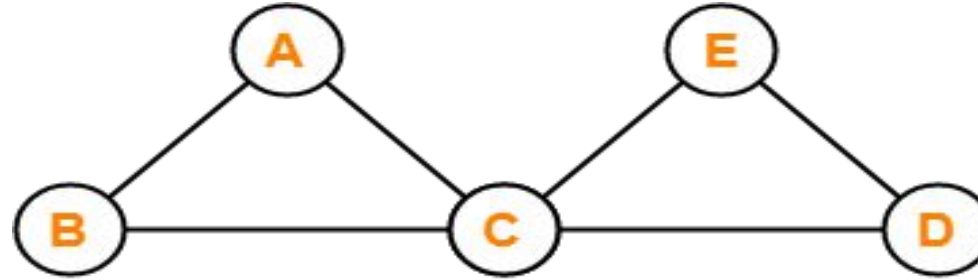
Example of Multi Graph

Pseudo graph: A graph G with a self-loop and some multiple edges is called a pseudo graph.



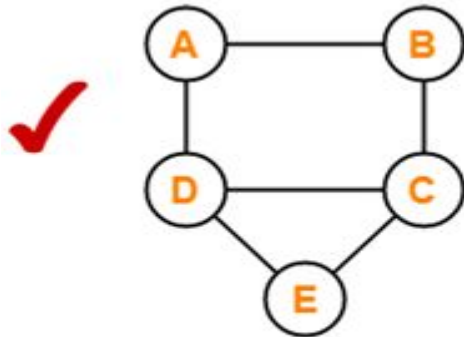
Basic structure (Graph types)

Euler Graph: Euler Graph is a connected graph in which all the vertices are even degree.

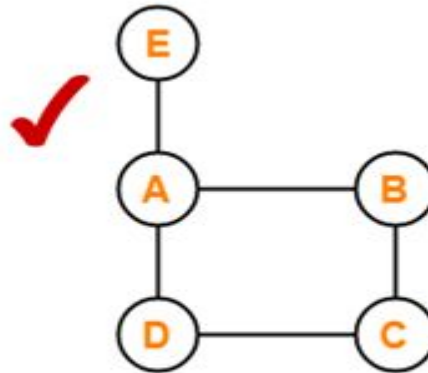


Example of Euler Graph

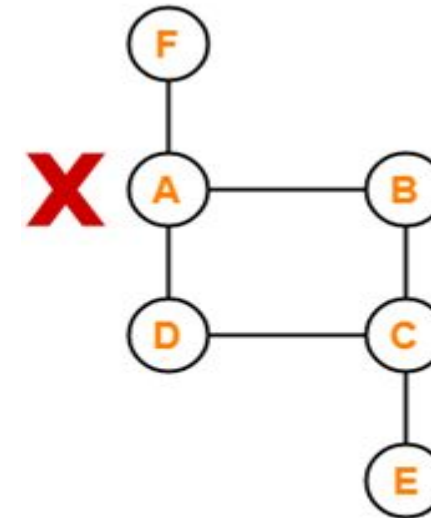
Hamiltonian Graph: A graph is said to be Hamiltonian graph if every vertex of the graph can be visited exactly once except starting vertex.



Hamiltonian Path = ABCDE



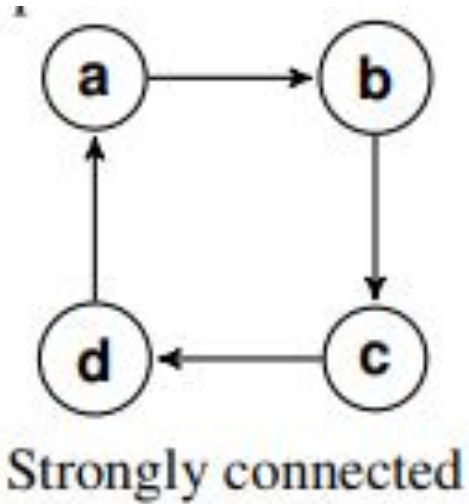
Hamiltonian Path = EABCD



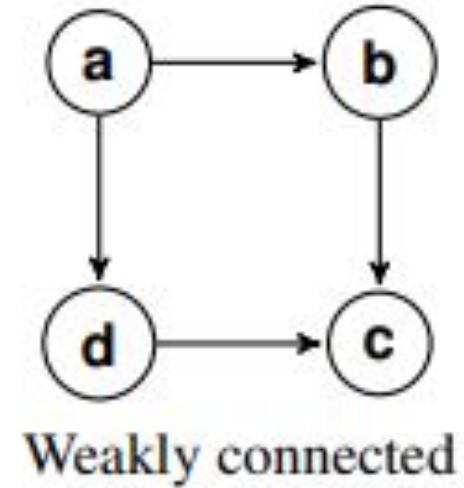
Hamiltonian Path Does Not Exist

Basic structure (Graph properties)

Strongly connected graphs are those in which every node can be reached from every other node directly or via.



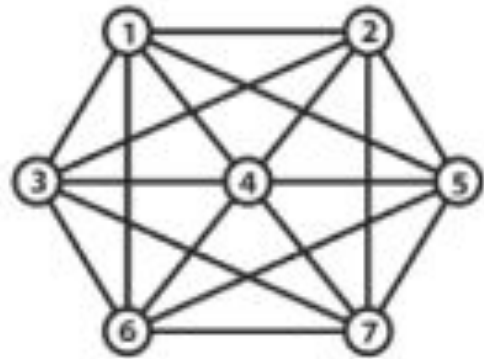
In a weakly connected graph, each node is not reachable from every other node directly or via.



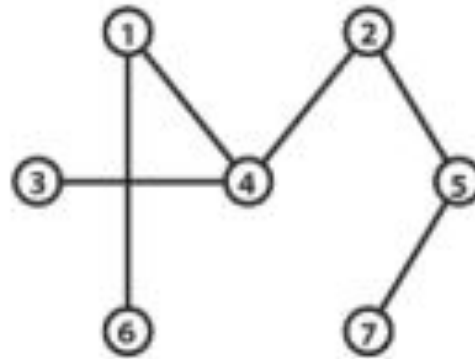
Basic structure (**Dense vs Sparse Graph**)

Dense graph is a graph in which the number of edges is close to the maximal number of edges.

Sparse graph is a graph in which the number of edges is close to the minimal number of edges.



Dense

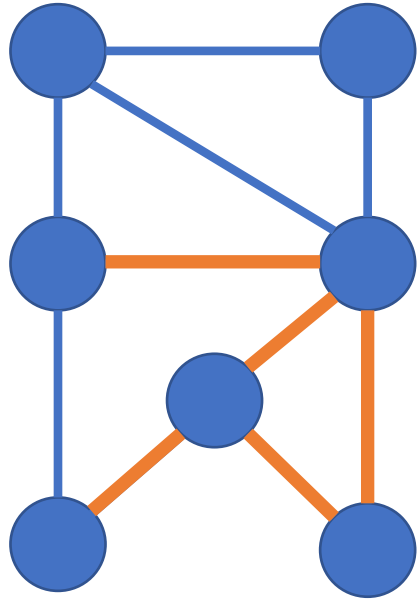


Sparse

Basic structure (Path types)

Walk:

1. Edges can be repeated (Multiple times)
2. Nodes can be repeated. (Multiple times)

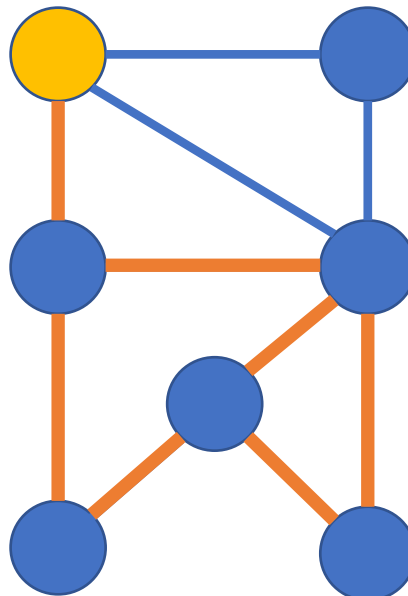


Walk

Closed walk:

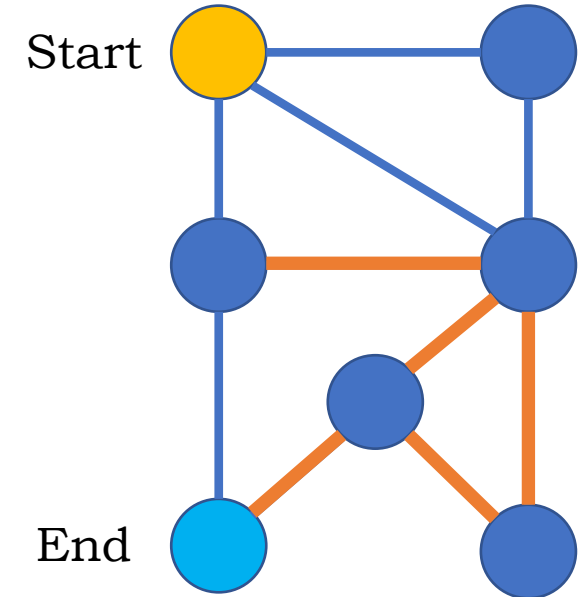
1. Edges can be repeated (Multiple times)
2. Nodes can be repeated. (Multiple times)
3. starts and ends nodes are the same.

Start and End



Open Walk:

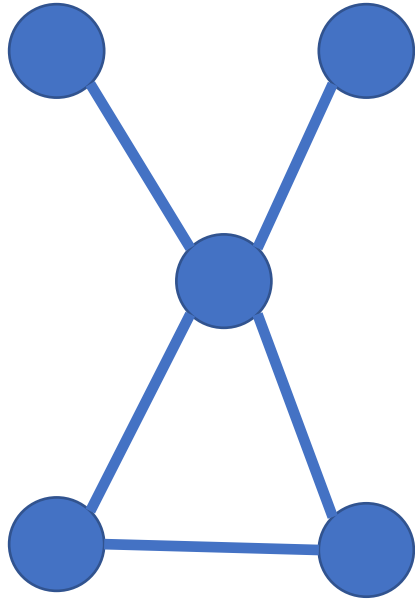
1. Edges can be repeated (Multiple times)
2. Nodes can be repeated. (Multiple times)
3. starts and ends nodes are different.



Basic structure (Path types)

Trail:

1. Edges can not be repeated
2. Nodes can be repeated. (Multiple times)
3. Start and end nodes may or may not be the same.

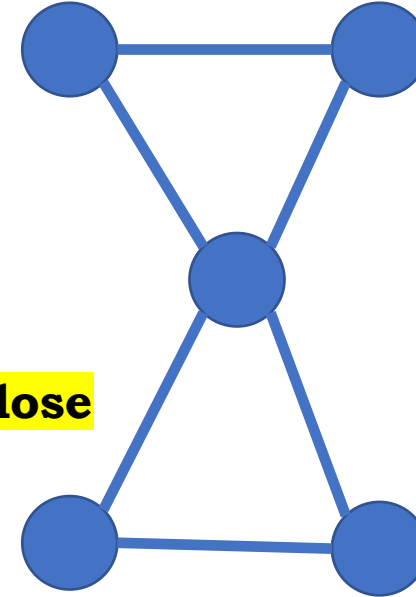


Trail

Circuit/trail close: is a trail

1. Edges can not be repeated
2. Nodes can be repeated. (Multiple times)
3. Start and end nodes must be the same node.

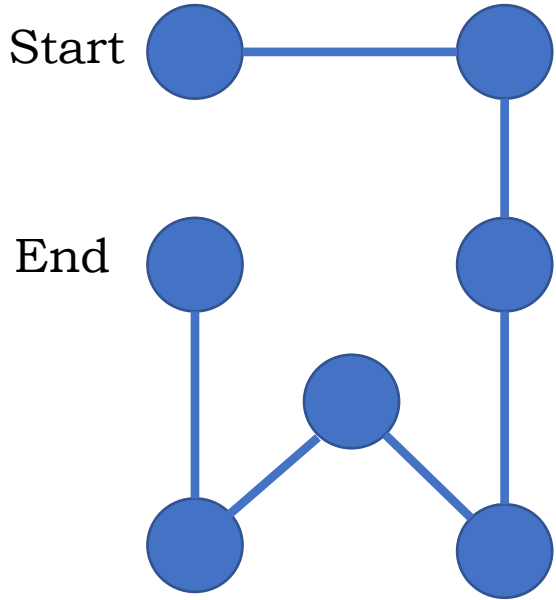
Circuit/trail close



Basic structure (Path types)

Path:

1. Edges can not be repeated
2. Nodes can not be repeated.

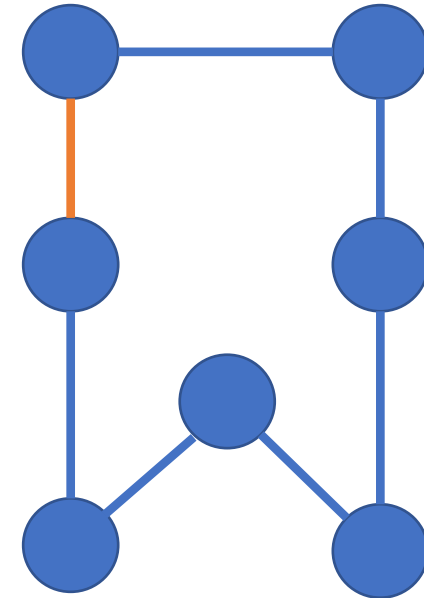


Path:

Cycle/ Close path: is a path

1. Edges can not be repeated
2. Nodes can not be repeated.
3. Start and end nodes must be the same

Start and End



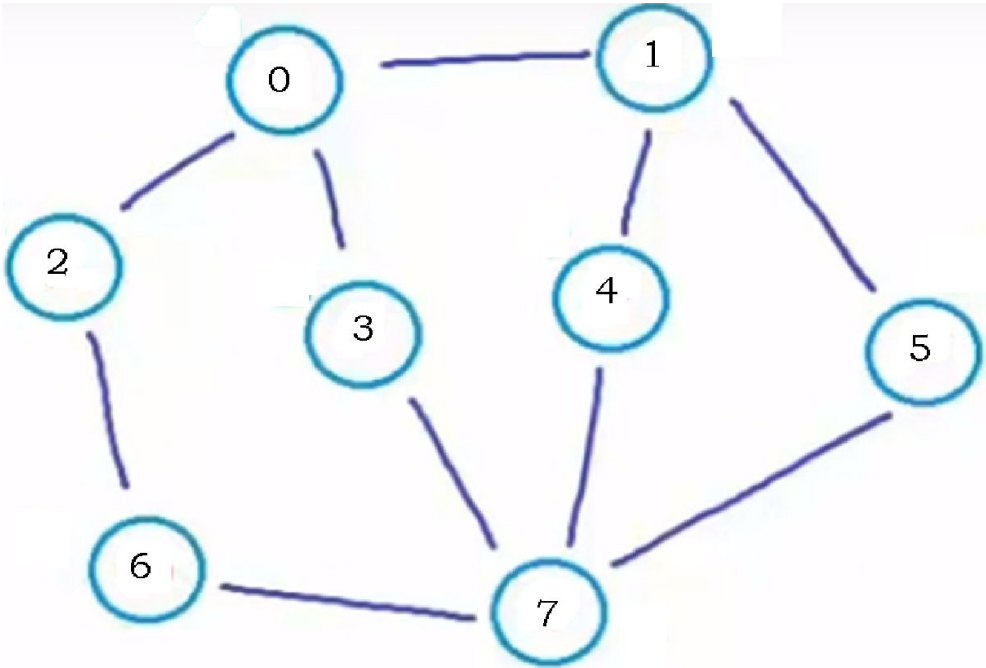
Cycle/ Close path

Basic structure (Graph Representation)

There are two ways to represent a graph.

1. Adjacency Matrix
2. Adjacency List

An adjacency matrix is a square matrix which describes a graph by representing which vertices are adjacent to which other vertices. Row and column must be equal to the number of vertex of a graph.



Adjacency Matrix



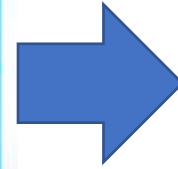
Adjacency Matrix

	0	1	2	3	4	5	6	7
0	0	1	1	1	0	0	0	0
1	1	0	0	0	1	1	0	0
2	1	0	0	0	0	0	1	0
3	1	0	0	0	0	0	0	1
4	0	1	0	0	0	0	0	1
5	0	1	0	0	0	0	0	1
6	0	0	1	0	0	0	0	1
7	0	0	0	1	1	1	1	0

Basic structure (Graph Representation)

Adjacency Matrix

	0	1	2	3	4	5	6	7
0	0	1	1	1	0	0	0	0
1	1	0	0	0	1	1	0	0
2	1	0	0	0	0	0	1	0
3	1	0	0	0	0	0	0	1
4	0	1	0	0	0	0	0	1
5	0	1	0	0	0	0	0	1
6	0	0	1	0	0	0	0	1
7	0	0	0	1	1	1	1	0

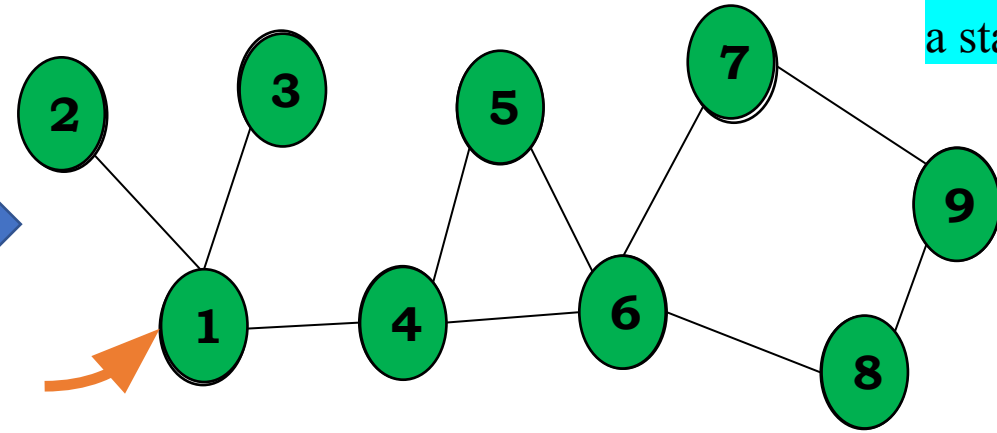
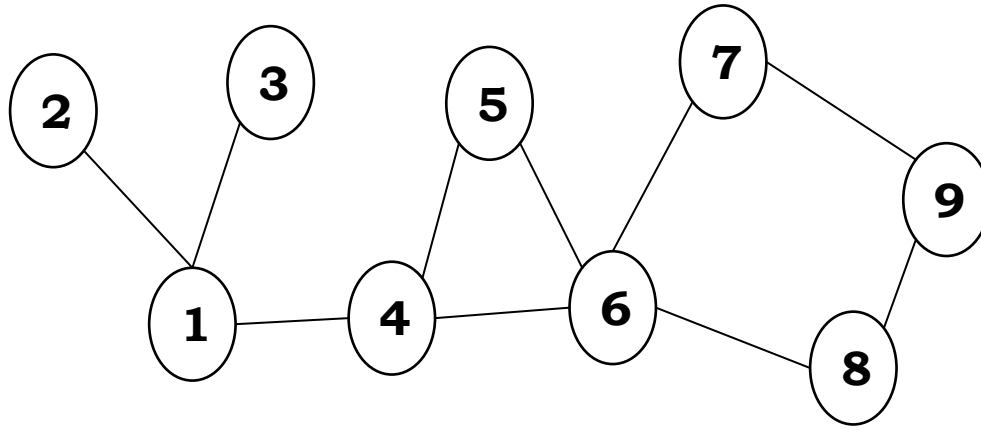


Adjacency List

0	1	2	3	
1	0	4	5	
2	0	6		
3	0	7		
4	1	7		
5	1	7		
6	2	7		
7	3	6	4	5

An adjacency list has a higher time complexity than Adjacency Matrixes, as they are accessed linearly, and a node can be connected to all nodes (N) at maximum. As a result, accessing an item on the adjacency list requires N comparisons. Therefore, the time complexity is $O(|N|)$. However, the space complexity is exactly equal to the number of edges. As a result, the space complexity is $O(|V| + |E|)$, where E represents an edge.

Breadth First Search or **BFS** for a Graph



Select any vertex as a starting node

BFS Search (Visited): 1 2 3 4 5 6 8 7 9

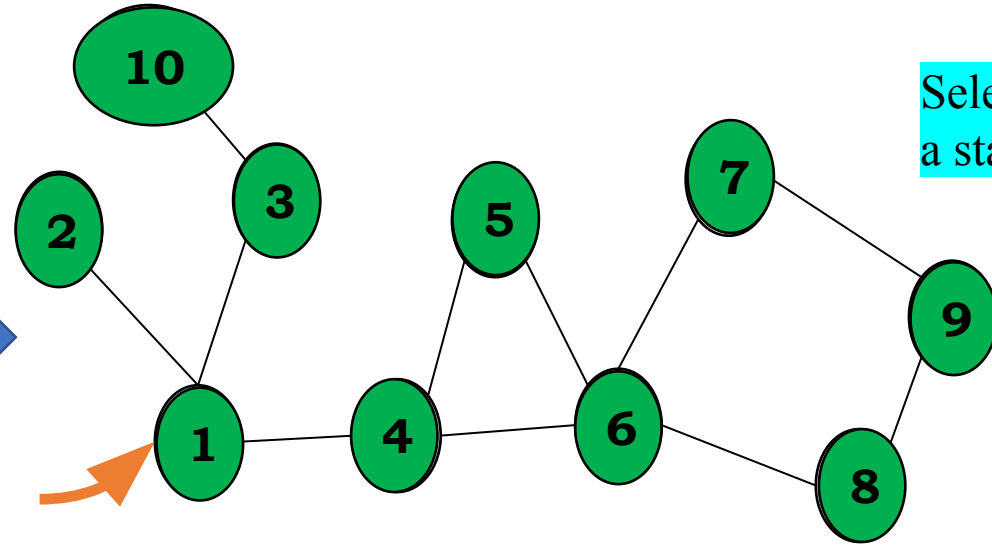
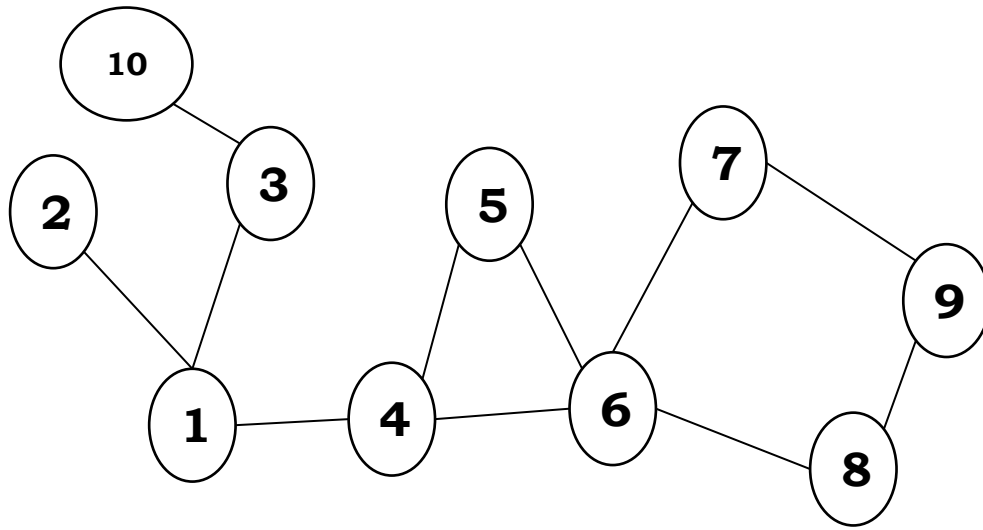
Queue

~~1~~ ~~2~~ ~~3~~ ~~4~~ ~~5~~ ~~6~~ ~~8~~ ~~7~~ ~~9~~

Insert the selected node 1 in queue.

Now, Insert all the Childs of the node 1 in queue in any order
For example, 2,3,4 or 4, 3,2 or 3,2,4 etc.

Depth First Search or **DFS** for a Graph



Select any vertex as a starting node

Insert any one of the Childs of the node 1 and put in stack.

DFS Search: **1 4 6 7 9 8 5 3 10 2**

The stack is now empty. The DFS traversal is complete. Depending on the node selected, traversing can differ.

~~8~~
~~9~~
~~7~~ ~~5~~
~~6~~ ~~10~~
~~4~~ ~~3~~ ~~2~~
~~1~~

Stack

Thank You